Week 2 Technical Report - Divide and Conquer

Project repository (complete source, tests, benchmarks and results):
https://github.com/RDeibel2025/CTX-Advanced-Algorithms

Robert Deibel · CSC 5300 Advanced Algorithms · Concordia University Texas · Fall 2026

📎 Complete project repository

https://github.com/RDeibel2025/CTX-Advanced-Algorithms

Fourteen Master Theorem solutions.

---

1. Executive Summary

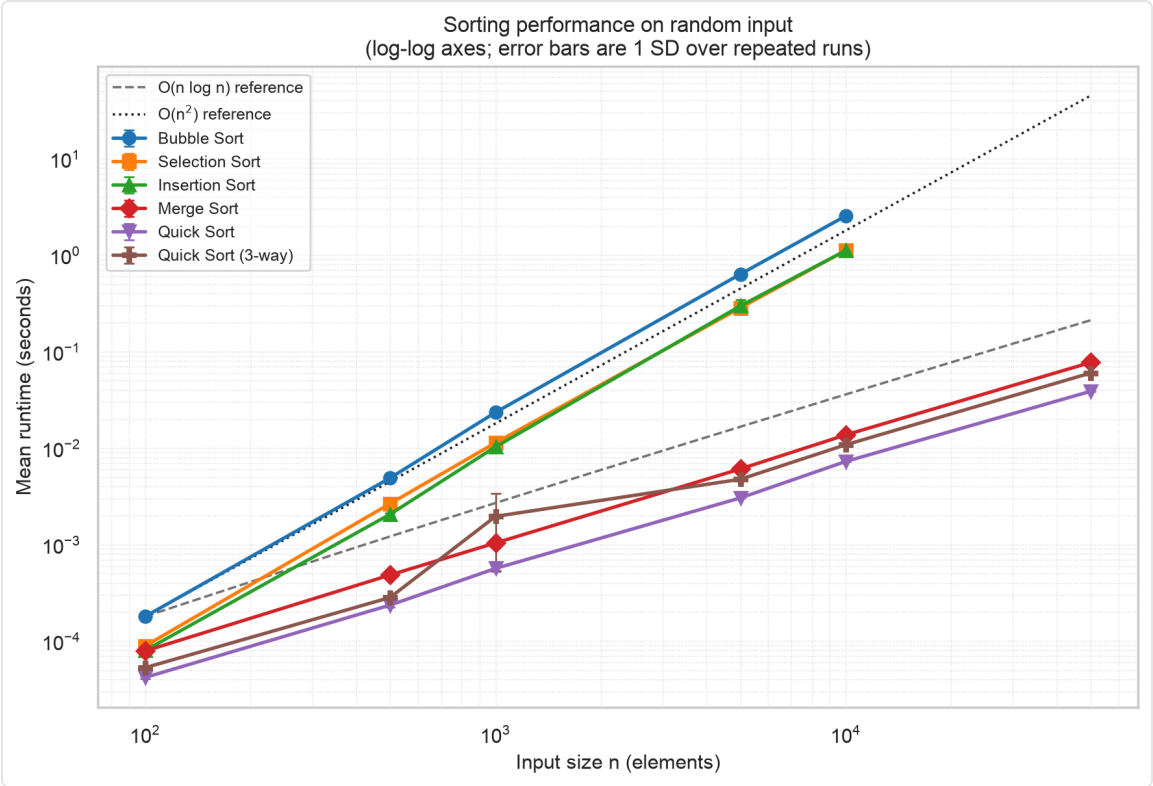
Merge sort and quicksort beat the Week 1 quadratics by a compounding factor - parity at n = 100, 154× at n = 10,000 - and both matched O(n log n), with doubling ratios of 2.03-2.20 and 2.15-2.39 against a predicted ~2.1. Quicksort's O(n^2) worst case never appeared. The most interesting result was negative: three-way partitioning rescues duplicate-heavy input only from a weakness Hoare's scheme lacks.

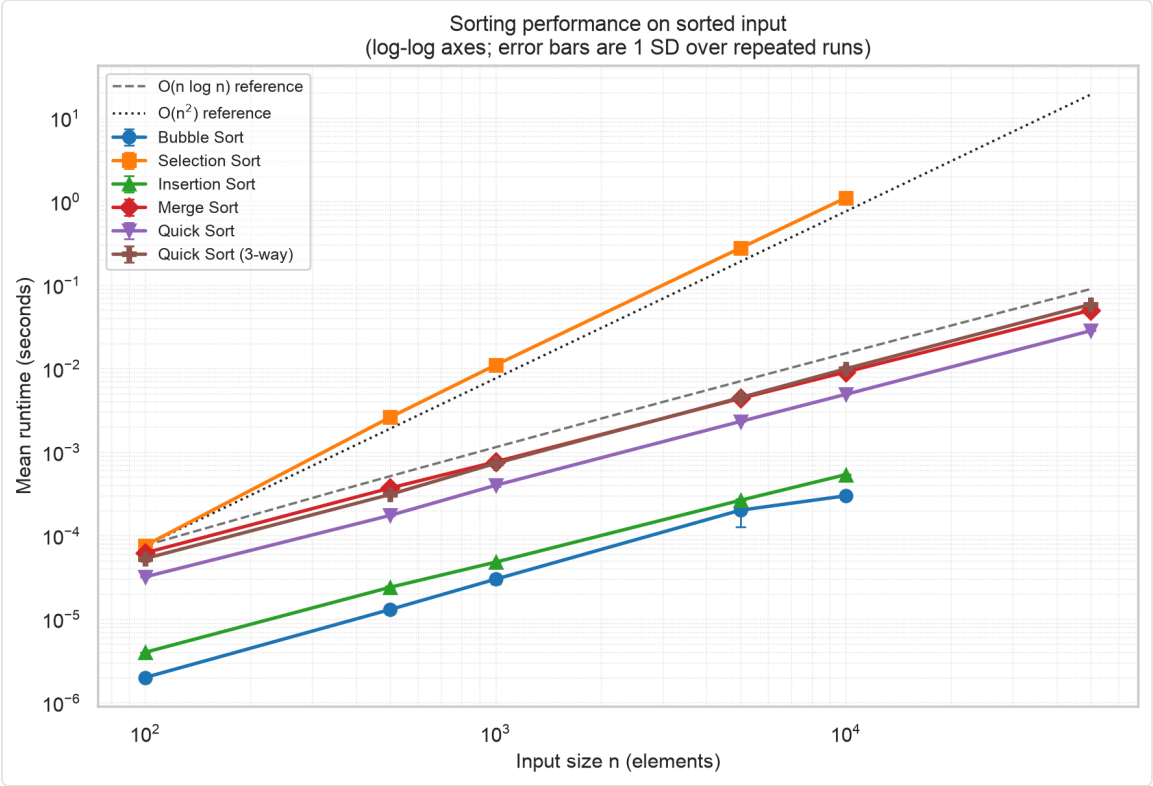
2. Methodology

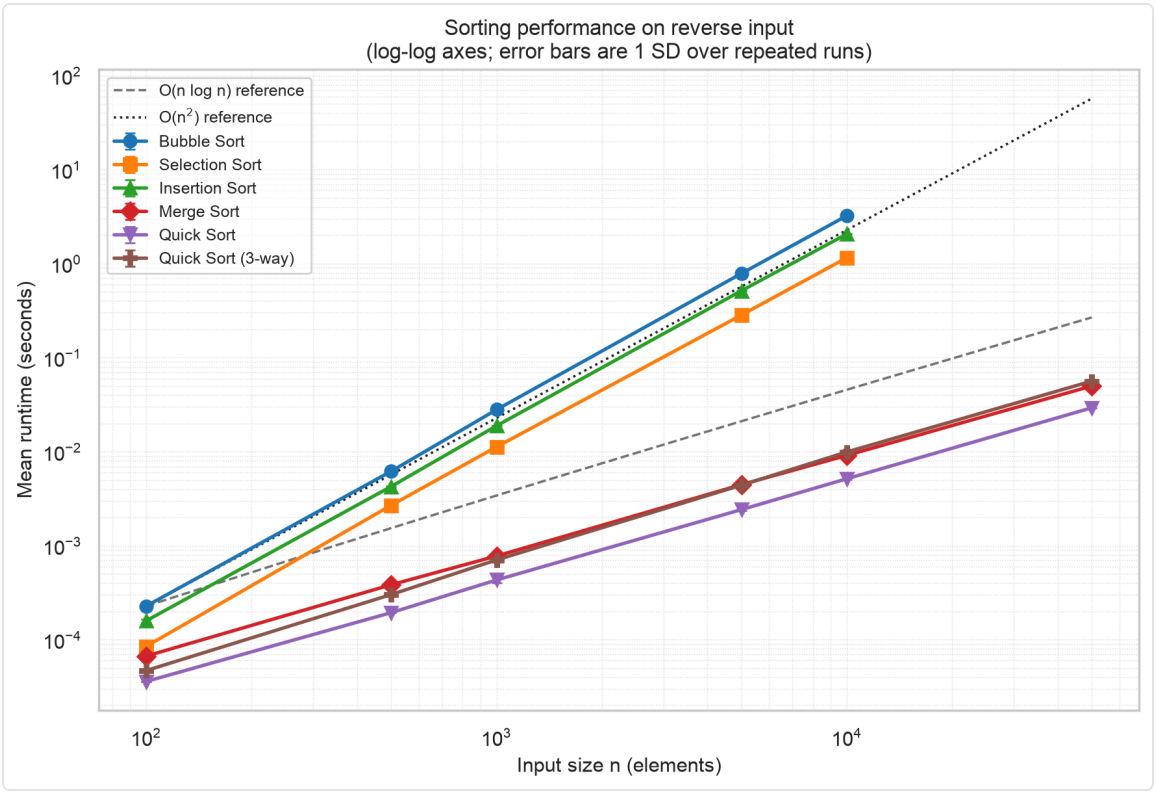
Apple M2 Max, 12 cores, 64 GB RAM; macOS 14.5 (arm64); CPython 3.12.4; timing by `time.perf_counter()`. Six algorithms (three Week 1 quadratics, merge, quicksort, quicksort three-way) over sizes 100 … 50,000 and six shapes: random, sorted, reverse, nearly sorted (5% transposed), many duplicates (10 distinct values), few unique (3). Two warm-up runs are discarded per cell, five measured below n = 10,000 and three above. Generation is seeded; every algorithm gets the identical list per cell, and every run is verified a sorted permutation. Median relative SD: 0.7%.

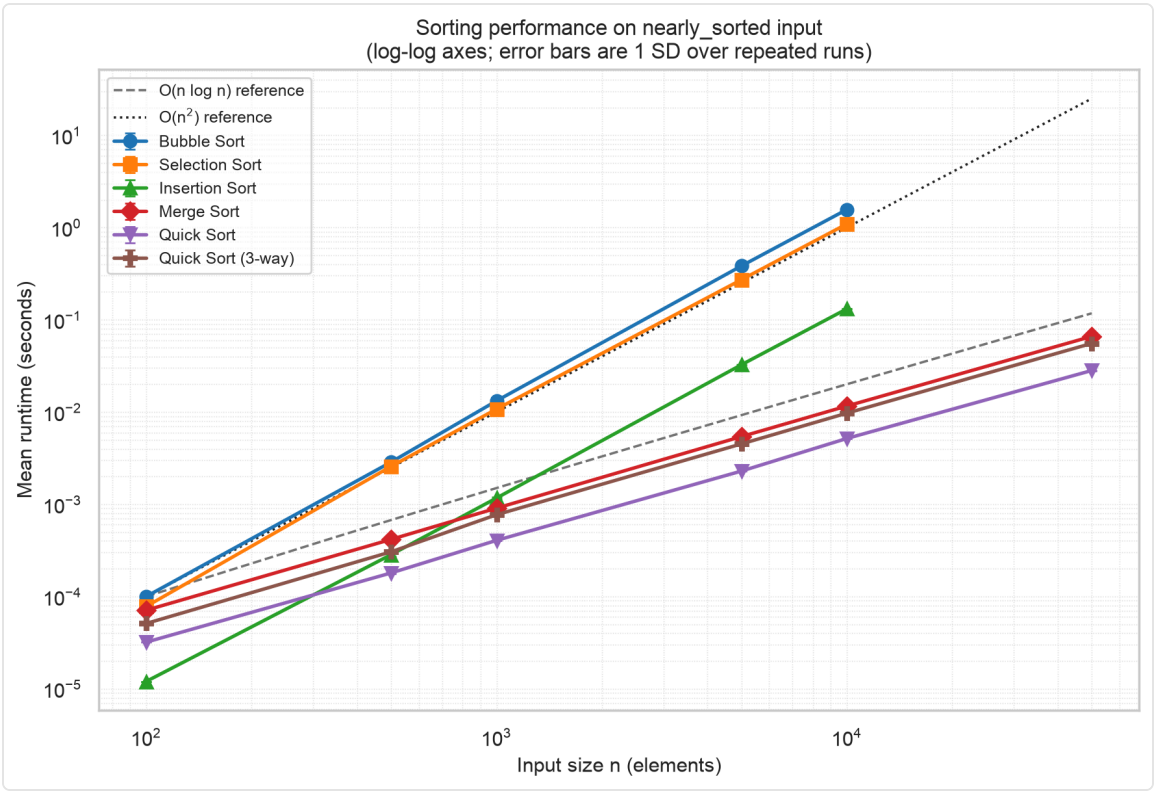
The O(n^2) size cap. The quadratics were measured only to n = 10,000; the eighteen cells at n = 50,000 appear in `comparison_table.csv` as `omitted` with reasons. Fitting $t = c \cdot n^2$ projects 64 s, 28 s and 28 s per run on random data - about an hour. I measured them anyway and checked the projection (§4).

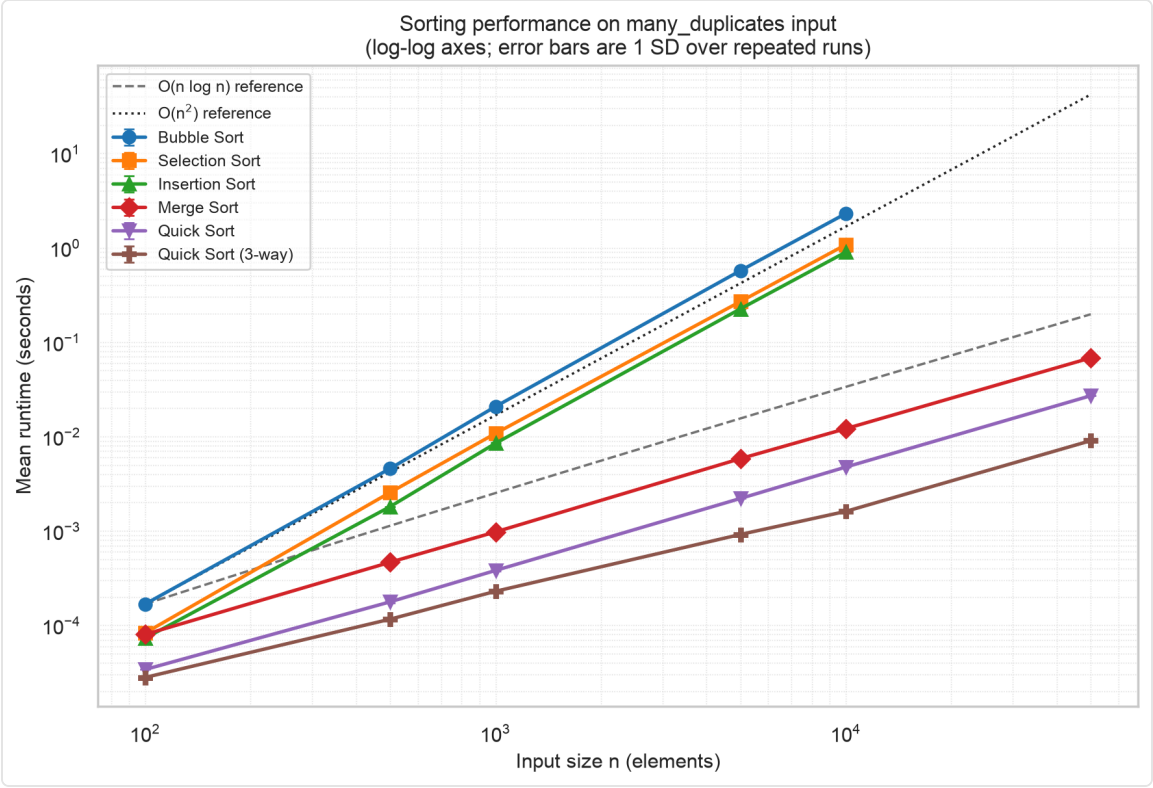
3. Results

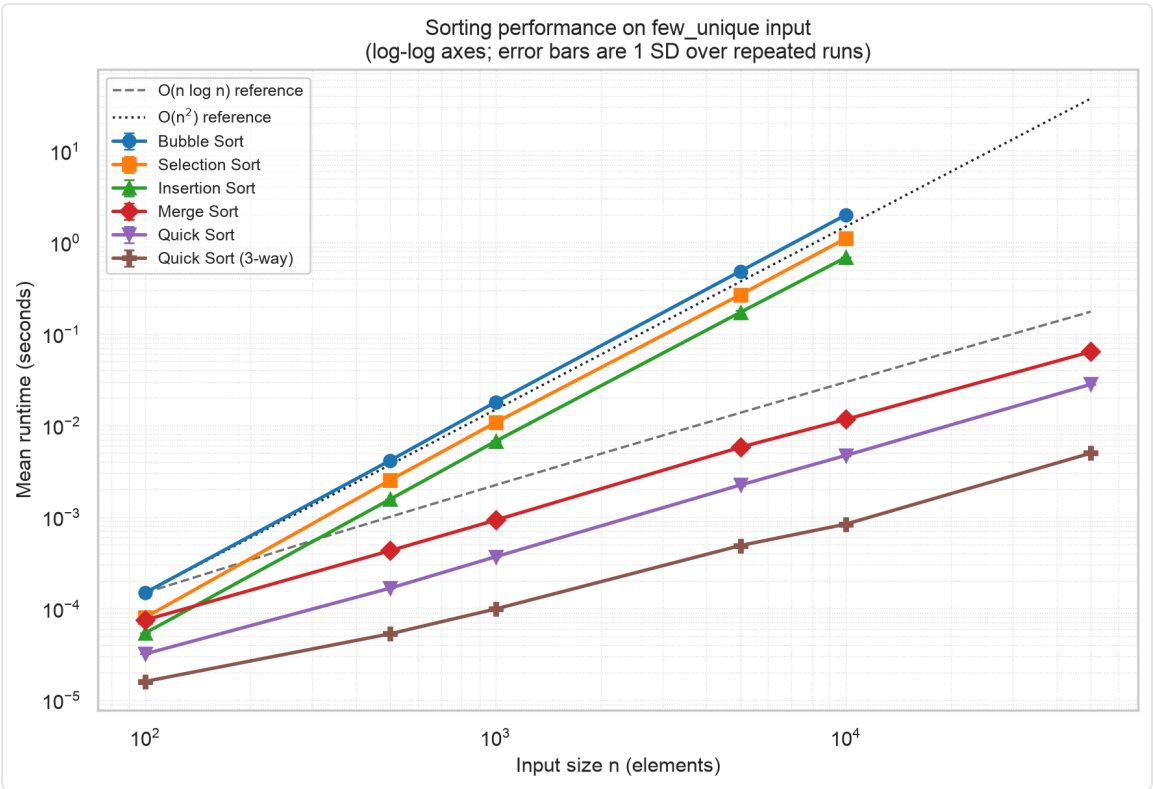

Sorting performance on random input
(log-log axes; error bars are 1 SD over repeated runs)
O(n log n) reference
O(n²) reference
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Quick Sort (3-way)
Mean runtime (seconds)
Input size n (elements)


Sorting performance on sorted input
(log-log axes; error bars are 1 SD over repeated runs)
O(n log n) reference
O(n²) reference
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Quick Sort (3-way)
Mean runtime (seconds)
Input size n (elements)


Sorting performance on reverse input
(log-log axes; error bars are 1 SD over repeated runs)
O(n log n) reference
O(n²) reference
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Quick Sort (3-way)
Mean runtime (seconds)
Input size n (elements)


Sorting performance on nearly_sorted input
(log-log axes; error bars are 1 SD over repeated runs)
O(n log n) reference
O(n²) reference
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Quick Sort (3-way)
Mean runtime (seconds)
Input size n (elements)





Mean seconds at n = 10,000:

Algorithm	random	sorted	reverse	nearly	many dup	few uniq
Bubble	2.5498	0.0003	3.2360	1.5665	2.3059	1.9940
Selection	1.1182	1.1031	1.1556	1.0929	1.0782	1.1047
Insertion	1.1167	0.0005	2.0667	0.1313	0.9004	0.6855
Merge	0.0137	0.0091	0.0091	0.0116	0.0121	0.0116
Quick	0.0073	0.0049	0.0052	0.0052	0.0047	0.0047
Quick 3-way	0.0108	0.0099	0.0099	0.0097	0.0016	0.0008

Speedup over the best quadratic:

n	100	500	1,000	5,000	10,000
Merge	1.0×	4.3×	9.9×	46.7×	81.5×
Quick	1.9×	8.7×	18.2×	93.2×	153.6×

The gap compounds; at n = 100 insertion and merge sort are indistinguishable (0.000080 s vs 0.000079 s, inside the run-to-run variation). Asymptotic superiority is a claim about large n.

Merge sort was among the most consistent - 1.5× variation across the six shapes at n = 10,000, against 10,823× for bubble and 3,856× for insertion. Its positional split gives the same recursion tree whatever the input order: no best or worst case to find. The price is O(n) auxiliary space.

QuickSort was fastest on five of six shapes, spread 1.5×: best on few-unique (0.0047 s), worst on random (0.0073 s) - nothing like the O(n log n)-to-O(n^2) range theory permits.

Surprising finding. Three-way partitioning is *slower* than two-way wherever values are mostly distinct - roughly half speed - paying off only on duplicate-heavy input: 2.9× at ten distinct values, 5.6× at three.

4. Complexity Validation

Observed t(2n)/t(n), mean of the 500→1,000 and 5,000→10,000 steps:

Algorithm	random	sorted	reverse	few uniq	Predicted
Bubble	4.40	1.90	4.32	4.23	4 / 2 adaptive
Selection	4.11	4.11	4.13	4.19	4
Insertion	4.38	2.01	4.22	4.14	4 / 2 adaptive
Merge	2.20	2.07	2.03	2.08	~2.1
Quick	2.39	2.20	2.18	2.15	~2.1

Merge sort stays within 0.10 of prediction everywhere. Bubble and insertion drop to ~2.1 on sorted input, their early exit appearing as a change of complexity class.

Did I observe quicksort's O(n²) worst case? No - every shape stayed at or below 2.4, including the three meant to be adversarial. Three mechanisms prevent it: randomized pivots leave no fixed input adversarial, so the worst case needs an unlucky *sequence of draws*; Hoare's partition splits a run of equal elements near its middle rather than onto one side; and recursing into the smaller partition caps stack depth at O(log n) - 10-14 frames across the six shapes at n = 100,000, against Python's 1,000-frame limit.

Unreachable *by this implementation* is not unreachable. Lomuto's partition, the scheme most often taught, reaches it at once:

Scheme (3 values)	growth/doubling	t(16,000)
Lomuto (2-way)	**3.98**	1.9246 s
Hoare (2-way)	2.15	0.0079 s
Dutch flag (3-way)	2.02	0.0016 s

3.98 against a predicted 4 is the worst case, measured - 244× slower than Hoare on identical input.

The capped cells. Measuring all eighteen put the fitted t = c·n² projection within **2.7%** on the sixteen genuinely quadratic cells (mean 1.4%), and off by **+436%** on the two sorted-input cells, where these algorithms are linear and n² is wrong. The extrapolation holds where its assumption does.

5. Optimization Impact

Insertion-sort cutoff, measured by varying the threshold. On random input at n = 50,000, disabling it costs 0.0563 s; a threshold of 10 gives 0.0389 s (**1.44×**) and the optimum of 20 gives 0.0353 s (1.59×). Past 20 the curve turns back up as insertion sort's O(k²) reasserts itself; on nearly-sorted and few-unique input it was still improving at 80, so one constant is a compromise.

Three-way partitioning: 5.6× at three distinct values, 2.9× at ten, ~2× slower otherwise. **Randomized pivot** cannot be isolated by timing; its evidence is §4's negative result.

6. Practical Recommendations

- **n below ~100:** insertion sort - it matches merge sort.
- **General purpose:** quicksort - randomized pivot, Hoare partition, insertion cutoff.
- **Stability or a guarantee:** merge sort, 1.9× slower on random.
- **Duplicate-heavy data:** three-way partitioning.
- **Never use Lomuto without a duplicate guard.**

7. Conclusion

Measurement matched theory closely enough that the interesting results were where it did not: asymptotic advantage is real but only past a measurable crossover, quicksort's worst case belongs to the partition scheme rather than to quicksort, and three-way partitioning is a trade.

8. References

- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed., Ch. 2.3-2.4, 4, 7
- Amakobe. *Advanced Computational Algorithms*, 2nd ed., Ch. 2
- Sedgewick, Wayne. *Algorithms*, 4th ed., §2.3

AI use acknowledgement

I used Anthropic's Claude (Claude Code) to draft the code, tests and first draft of this report, from a specification I wrote off the instructions and assigned reading. The measurements are my own, every figure above computed from the result CSVs rather than transcribed. The design decisions were mine - Hoare's partition over Lomuto's, recursing into the smaller partition, and measuring the capped cells rather than projecting them. Full disclosure: `docs/AI_USE.md`.

Appendix A: Master Theorem - Recurrence Relations

Robert Deibel
CSC 5300 Advanced Algorithms · Concordia University Texas · Fall 2026
Week 2 - Divide and Conquer Implementation Project

Repository: https://github.com/RDeibel2025/CTX-Advanced-Algorithms

Fourteen recurrences solved below. Ten are straightforward applications of the Master Theorem; three are included precisely because the theorem does **not** settle them (§9, §13 and §14), since knowing where a tool stops working is part of knowing how to use it. The remaining one, §11, falls in the gap between cases 2 and 3 and needs the *extended* case 2 rather than the three-case statement. That is 10 + 3 + 1 = 14.

The theorem, as used here

For a recurrence of the form

> **T(n) = a·T(n/b) + f(n)**, with a ≥ 1, b > 1

compare f(n) against the *watershed function* n^(log_b a). That exponent is what the recursion costs on its own - it counts the leaves of the recursion tree - and the three cases ask whether the work done splitting and combining is smaller, equal, or larger than that.

Case	Condition	Result
1	f(n) = O(n^(log_b a – ε)) for some ε > 0	Θ(n^(log_b a)) - leaves dominate
2	f(n) = Θ(n^(log_b a) · log^k n), k ≥ 0	Θ(n^(log_b a) · log^(k+1) n) - balanced
3	f(n) = Ω(n^(log_b a + ε)) for some ε > 0, **and** a·f(n/b) ≤ c·f(n) for some c < 1 and large n	Θ(f(n)) - the root dominates

Two requirements are easy to skip past, and they are what §11 and §13 fall foul of. Cases 1 and 3 need f(n) to differ from the watershed by a *polynomial* factor - an n^ε gap, not merely a logarithmic one. And case 3 additionally needs the **regularity condition**, which is what rules out pathologically oscillating f.

---

1. Merge sort - T(n) = 2T(n/2) + Θ(n)

The recurrence implemented in `src/sorting/merge_sort.py`: two recursive calls on half the input, plus a linear merge.

a	2
b	2
f(n)	Θ(n)
log_b a	$\log_2 2 = \mathbf{1}$
watershed	$n^1 = n$

f(n) = Θ(n) = Θ(n^(log_b a) · $\log^0$ n), so **case 2 with k = 0**.

> **T(n) = Θ(n log n)**

Every level of the recursion does Θ(n) work merging, and there are $\log_2$ n levels - the cost is spread evenly rather than concentrated at the root or the leaves. Because the split is positional it is always balanced, so this recurrence describes merge sort's best, average *and* worst case. The measured doubling ratios of 2.03-2.20 tabulated in `week2_report.md` §4 - one per data shape, each the mean of the two doubling steps - are this result observed rather than derived.

---

2. Binary search - T(n) = T(n/2) + Θ(1)

a	1
b	2
f(n)	Θ(1)
log_b a	$\log_2 1 = \mathbf{0}$
watershed	$n^0 = 1$

f(n) = Θ(1) = Θ(n^0 · $\log^0$ n), so **case 2 with k = 0**.

> **T(n) = Θ(log n)**

Only one subproblem survives each step, so the recursion tree is a path rather than a tree. The Θ(1) comparison at each node is exactly the watershed, and the log factor comes from the depth.

---

3. Tree traversal - T(n) = 2T(n/2) + Θ(1)

a	2
b	2
f(n)	Θ(1)
log_b a	**1**
watershed	n

Θ(1) is polynomially *smaller* than n - take ε = 1, giving f(n) = O(n^(1−1)) = O(1). **Case 1.**

> **T(n) = Θ(n)**

The leaves dominate. This is the useful contrast with §1: the same 2T(n/2) split costs Θ(n) when the per-node work is constant and Θ(n log n) when it is linear. All the extra cost in merge sort comes from the merge, not from the recursion.

---

4. Karatsuba multiplication - T(n) = 3T(n/2) + Θ(n)

a	3
b	2
f(n)	Θ(n)
log_b a	$\log_2 3 \approx$ **1.585**
watershed	n^1.585

f(n) = Θ(n) = O(n^(1.585 − ε)) with ε = 0.585. **Case 1.**

> **T(n) = Θ(n^$\log_2$3) ≈ Θ(n^1.585)**

Karatsuba's insight is arithmetic, not structural: the naive algorithm needs four half-size multiplications, and it computes the same result with three, using extra additions. Those additions are the Θ(n) term and they are dominated. Reducing a from 4 to 3 is what beats Θ(n^2).

---

5. The naive version - T(n) = 4T(n/2) + Θ(n)

a	4
b	2
f(n)	Θ(n)
log_b a	$\log_2 4 = \mathbf{2}$
watershed	n^2

f(n) = Θ(n) = O(n^(2 − ε)) with ε = 1. **Case 1.**

> **$T(n) = \Theta(n^2)$**

Worth solving next to §4 because the two differ only in a. Four recursive calls buy nothing over schoolbook long multiplication; three beat it. The recursion structure is identical, so the entire improvement is attributable to that one parameter.

---

6. Strassen's matrix multiplication - T(n) = 7T(n/2) + Θ(n²)

a	7
b	2
f(n)	$\Theta(n^2)$
log_b a	$\log_2 7 \approx \mathbf{2.807}$
watershed	n^2.807

f(n) = Θ(n²) = O(n^(2.807 − ε)) with ε ≈ 0.807. **Case 1.**

> **$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$**

Same trade as Karatsuba, one dimension up. Eight submatrix products (a = 8, $\log_2 8 = 3$) reproduce the $\Theta(n^3)$ of the standard algorithm; Strassen finds seven, and the extra matrix additions cost $\Theta(n^2)$, which is polynomially below the watershed and therefore free.

---

7. Standard matrix multiplication - $T(n) = 8T(n/2) + \Theta(n^2)$

a	8
b	2
f(n)	$\Theta(n^2)$
log_b a	$\log_2 8 = \mathbf{3}$
watershed	n^3

f(n) = $\Theta(n^2)$ = O(n^(3 − ε)) with ε = 1. **Case 1.**

> **$T(n) = \Theta(n^3)$**

The baseline §6 improves on, confirming that block recursion by itself buys nothing.

---

8. Quicksort's balanced case - $T(n) = 2T(n/2) + \Theta(n)$

Identical in form to §1, and worth stating separately because in quicksort this recurrence is a *hope* rather than a guarantee. The partition is Θ(n) and the split is even only when the pivot lands near the median.

a	2, b

Case 2, so:

> **$T(n) = \Theta(n \log n)$**

Randomized pivot selection does not make this recurrence hold; it makes it hold *in expectation*, where §1 delivers it deterministically. That distinction did not show up in the measured spread across input shapes - merge sort varied by 1.51× at n = 10,000 and quicksort by 1.54×, which is randomization delivering in practice what merge sort guarantees in principle. Where it does show up is the tail: change the partition scheme and quicksort reaches §9 instead, as the Lomuto study did.

---

9. Quicksort's worst case - T(n) = T(n−1) + Θ(n)

a	1
b	-
f(n)	Θ(n)

The Master Theorem does not apply. The theorem requires subproblems of size n/b for a constant b > 1 - a *fixed fraction* of the input. Here the subproblem shrinks by a constant *amount*, so there is no b to speak of.

Solve by direct expansion instead:

T(n) = n + (n−1) + (n−2) + … + 1 = n(n+1)/2

> **T(n) = Θ(n²)**

This is the recurrence that arises when every partition puts all remaining elements on one side. Section 4 of the report records that it was never observed for the production quicksort across any of the six data shapes - but the Lomuto partition study reached it exactly, with a measured growth ratio of 3.98 per doubling against the predicted 4.

---

10. Binary search variant - T(n) = 2T(n/2) + Θ(log n)

a	2
b	2
f(n)	Θ(log n)
log_b a	**1**
watershed	n

Θ(log n) is polynomially smaller than n: for ε = 0.5, log n = O(n^0.5). **Case 1.**

> **T(n) = Θ(n)**

A useful check on intuition - a logarithmic combine step is asymptotically invisible against a linear watershed, so this costs no more than §3 with its Θ(1) combine.

---

11. Case 2 extended - T(n) = 2T(n/2) + Θ(n log n)

a	2
b	2
f(n)	Θ(n log n)
log_b a	**1**
watershed	n

This one needs care. f(n) = Θ(n log n) is **larger** than the watershed n, which looks like case 3 - but case 3 requires f(n) = Ω(n^(1+ε)) for some ε > 0, and n log n is *not* polynomially larger than n. It grows faster by a logarithmic factor only, and log n = o(n^ε) for every ε > 0. So the **three-case Master Theorem as usually stated does not cover this recurrence**; it falls in the gap between cases 2 and 3.

The *extended* case 2 does cover it. In the form f(n) = Θ(n^(log_b a) · log^k n) with k = 1:

> **T(n) = Θ(n^(log_b a) · log^(k+1) n) = Θ(n log² n)**

Confirmed by the recursion tree: $\log_2 n$ levels, and level i does 2^i · (n/2^i) · log(n/2^i) = n · log(n/2^i) work. Summing over i = 0 … log n − 1 gives n · Σ(log n − i) = n · Θ(log² n).

The lesson is that "f is bigger than the watershed" is not enough for case 3. The gap has to be polynomial.

---

12. Case 3, with the regularity condition checked - T(n) = 2T(n/2) + Θ(n²)

a	2
b	2
f(n)	Θ(n²)
log_b a	**1**
watershed	n

f(n) = Θ(n²) = Ω(n^(1+ε)) with ε = 1 - a genuine polynomial gap, unlike §11. Case 3 also requires regularity, which most treatments assert and skip; here it is checked:

> a·f(n/b) = 2·(n/2)² = 2·n²/4 = n²/2 = **0.5·f(n)** ≤ c·f(n) with c = 0.5 < 1 ✓

Both conditions hold. **Case 3.**

> **$T(n) = \Theta(n^2)$**

The root dominates so completely that the entire recursion below it is a constant-factor detail. The total is a geometric series in which each level costs half the one above, summing to at most 2·f(n).

---

13. Where the Master Theorem fails - T(n) = 2T(n/2) + n/log n

a	2
b	2
f(n)	n / log n
log_b a	**1**
watershed	n

f(n) = n/log n is **smaller** than n, which suggests case 1. But case 1 requires f(n) = O(n^(1−ε)) for some fixed ε > 0, and that fails: for any ε > 0,

> lim (n→∞) [n/log n] / n^(1−ε) = lim n^ε / log n = ∞

so n/log n is not O(n^(1−ε)) for any ε. It is smaller than the watershed by a logarithmic factor only, never a polynomial one - the mirror image of the §11 problem. Case 2 does not apply either, since n/log n = n·log^(−1) n requires k = −1 and the extended case 2 requires k ≥ 0.

The recurrence falls between cases 1 and 2, and the Master Theorem returns no answer at all.

It is still solvable, just not with this tool. By the recursion tree: there are $\log_2 n$ levels, and level i does 2^i · (n/2^i)/log(n/2^i) = n / (log n − i) work. Summing over i = 0 … log n − 1:

T(n) = n · Σ(i=0 to log n−1) 1/(log n − i) = n · (1 + 1/2 + … + 1/log n) = n · H(log n)

where H is the harmonic series, H(m) = Θ(log m). Therefore:

> **$T(n) = \Theta(n \log \log n)$**

A bound strictly between the Θ(n) of case 1 and the Θ(n log n) of case 2 - which is exactly why no case could produce it. The Akra-Bazzi method handles this class of recurrence in general.

---

14. Fibonacci by naive recursion - T(n) = T(n−1) + T(n−2) + Θ(1)

a	2 subproblems, of *different* sizes
b	-

The Master Theorem does not apply, for two independent reasons: the subproblems shrink by a constant amount rather than a constant factor (as in §9), and they are not the same size as each other, which the theorem's a·T(n/b) form cannot express.

The recurrence is its own solution. T(n) exceeds the Fibonacci numbers themselves, which grow as $\phi^n/\sqrt{5}$ with $\phi = (1+\sqrt{5})/2$:

> **$T(n) = \Theta(\phi^n) \approx \Theta(1.618^n)$**

Included as the reminder that "divide and conquer" is not automatically efficient. Splitting a problem in two only helps when the pieces are *fractions* of the original; splitting into pieces of size n−1 and n−2 duplicates almost all the work, which is what memoisation eliminates to give Θ(n).

Summary

#	Recurrence	a	b	f(n)	log_b a	Case	Result
1	2T(n/2) + Θ(n)	2	2	Θ(n)	1	2	Θ(n log n)
2	T(n/2) + Θ(1)	1	2	Θ(1)	0	2	Θ(log n)
3	2T(n/2) + Θ(1)	2	2	Θ(1)	1	1	Θ(n)
4	3T(n/2) + Θ(n)	3	2	Θ(n)	1.585	1	Θ(n^1.585)
5	4T(n/2) + Θ(n)	4	2	Θ(n)	2	1	$\Theta(n^2)$
6	7T(n/2) + $\Theta(n^2)$	7	2	$\Theta(n^2)$	2.807	1	Θ(n^2.807)
7	8T(n/2) + $\Theta(n^2)$	8	2	$\Theta(n^2)$	3	1	$\Theta(n^3)$
8	2T(n/2) + Θ(n)	2	2	Θ(n)	1	2	Θ(n log n)
9	T(n−1) + Θ(n)	-	-	Θ(n)	-	**N/A**	$\Theta(n^2)$
10	2T(n/2) + Θ(log n)	2	2	Θ(log n)	1	1	Θ(n)
11	2T(n/2) + Θ(n log n)	2	2	Θ(n log n)	1	2 ext.	$\Theta(n \log^2 n)$
12	2T(n/2) + $\Theta(n^2)$	2	2	$\Theta(n^2)$	1	3	$\Theta(n^2)$
13	2T(n/2) + n/log n	2	2	n/log n	1	**N/A**	Θ(n log log n)
14	T(n−1) + T(n−2) + Θ(1)	-	-	Θ(1)	-	**N/A**	$\Theta(\phi^n)$

Three of the fourteen (§9, §13, §14) are outside the theorem's reach, for three different reasons: subproblems that shrink by a constant amount, a gap that is logarithmic rather than polynomial, and subproblems of unequal size.

References

- Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, 4th ed., Ch. 4 (the Master Theorem, its proof, and the Akra-Bazzi generalisation).
- Amakobe, *Advanced Computational Algorithms*, 2nd ed., Ch. 2.

Appendix B: Worked Example

Output of `examples/week2_demo.py`, captured by running it. The script demonstrates merge sort and quicksort on inputs small enough to read: the edge cases, the non-destructive contract, non-integer element types, three-way partitioning timed against two-way on duplicate-heavy input, stability, and a cross-check that all five algorithms agree. Every claim it prints is also asserted, so it exits non-zero if any of them stops holding.

```
======================================================================
CSC 5300 Advanced Algorithms — Week 2 demonstration
Merge sort and quicksort — Robert Deibel
======================================================================
  random seed              2026
  INSERTION_SORT_CUTOFF    10
  Every line printed below is also asserted; this script exits
  non-zero if any demonstrated claim fails to hold.

======================================================================
1. Sorting a small list
======================================================================
  input                [38, 27, 43, 3, 9, 82, 10]

  merge_sort           [3, 9, 10, 27, 38, 43, 82]
  quick_sort           [3, 9, 10, 27, 38, 43, 82]
  sorted() reference   [3, 9, 10, 27, 38, 43, 82]

  Both agree with Python's own sorted().

  merge() combines two already-sorted runs in linear time:
  left                 [3, 9, 27, 38]
  right                [10, 43, 82]
  merge(left, right)   [3, 9, 10, 27, 38, 43, 82]

======================================================================
2. Edge cases the caller should not have to handle
======================================================================
  case            input               merge_sort          quick_sort          quick_sort 3-way
  ------------------------------------------------------------------------------------------------
  empty           []                  []                  []                  []
  single element  [42]                [42]                [42]                [42]
  two, in order   [1, 2]              [1, 2]              [1, 2]              [1, 2]
  two, reversed   [2, 1]              [1, 2]              [1, 2]              [1, 2]
  all equal       [7, 7, 7, 7, 7]     [7, 7, 7, 7, 7]     [7, 7, 7, 7, 7]     [7, 7, 7, 7, 7]
  already sorted  [1, 2, 3, 4, 5]     [1, 2, 3, 4, 5]     [1, 2, 3, 4, 5]     [1, 2, 3, 4, 5]
  reverse sorted  [5, 4, 3, 2, 1]     [1, 2, 3, 4, 5]     [1, 2, 3, 4, 5]     [1, 2, 3, 4, 5]

  No special-casing by the caller; no crashes on empty input.

======================================================================
3. The caller's list is never modified
======================================================================
  before sorting       [5, 3, 8, 1, 9, 2]
  merge_sort returns   [1, 2, 3, 5, 8, 9]
  original afterwards  [5, 3, 8, 1, 9, 2]

  The input is byte-for-byte unchanged, and the result is a
  separate object — writing to it cannot affect the original:
  result[0] = 999      [999, 2, 3, 5, 8, 9]
  original still       [5, 3, 8, 1, 9, 2]

  quick_sort holds to the same contract, even though its
  partitioning genuinely happens in place — on a copy it owns.

======================================================================
4. Any comparable element type, not just integers
======================================================================
  integers             [1, 2, 3]
  negatives            [-9, -5, -3, -1]
  floats               [-0.5, 0.0, 2.25, 3.5]
  strings              ['apple', 'banana', 'fig', 'pear']
  tuples               [(1, 'z'), (2, 'a'), (2, 'b')]
```

```
  The algorithms only ever compare elements with < and >, so
  anything orderable works.

================================================================================
5. Three-way partitioning on duplicate-heavy input
================================================================================
  quick_sort takes a three_way flag. It splits into < = > rather
  than < >=, so every element equal to the pivot is finished in
  one pass and never looked at again.

  input                [5, 1, 5, 3, 5, 2, 5, 4, 5, 0, 5, 3, 1]
  two-way              [0, 1, 1, 2, 3, 3, 4, 5, 5, 5, 5, 5, 5]
  three-way            [0, 1, 1, 2, 3, 3, 4, 5, 5, 5, 5, 5, 5]

  Identical output. The difference is how much work each does.

  n = 20,000, only 3 distinct values, best of 3 runs:
    two-way   0.00974 s
    three-way 0.00185 s
    three-way is 5.3x faster here

  It is a trade, not a free improvement: on data where values
  are mostly distinct, three-way partitioning is the slower of
  the two. See analysis/week2_report.md section 5.

================================================================================
6. Stability: merge sort preserves the order of equal keys
================================================================================
  Records carry a key and a tag. They are compared by key only,
  so the tags show what happened to elements that tied.

  keys                 [1, 0, 2, 1, 1, 2, 0, 1, 2, 2, 2]
  tags in              [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

  merge_sort keys      [0, 0, 1, 1, 1, 1, 2, 2, 2, 2, 2]
  merge_sort tags      [1, 6, 0, 3, 4, 7, 2, 5, 8, 9, 10]
    -> equal keys kept their input order: merge sort is stable.

  quick_sort keys      [0, 0, 1, 1, 1, 1, 2, 2, 2, 2, 2]
  quick_sort tags      [6, 1, 0, 3, 4, 7, 2, 5, 8, 9, 10]
    -> 2 positions differ from the stable order:
       quicksort put [6, 1], a stable sort would put [1, 6].
       Both are correctly sorted by key. Quicksort simply makes
       no promise about ties, because partitioning swaps
       non-adjacent elements.

  Note the size: 11 elements. Quicksort finishes any range of
  10 or fewer with insertion sort, which is stable, so below
  11 elements it never partitions and cannot reorder a tie.

================================================================================
7. All five algorithms agree
================================================================================
  200 random integers, sorted by each of the five algorithms.
  Every output is identical and matches sorted():

    bubble_sort      first 8: [-50, -50, -50, -49, -49, -49, -47, -47]
    selection_sort   first 8: [-50, -50, -50, -49, -49, -49, -47, -47]
    insertion_sort   first 8: [-50, -50, -50, -49, -49, -49, -47, -47]
    merge_sort       first 8: [-50, -50, -50, -49, -49, -49, -47, -47]
    quick_sort       first 8: [-50, -50, -50, -49, -49, -49, -47, -47]

  The Week 1 and Week 2 implementations are interchangeable.
  tests/test_sorting_comparison.py checks this exhaustively.

================================================================================
All demonstrations held. See also:
  analysis/week2_report.md        the performance study
  analysis/week2_recurrences.md   Master Theorem solutions
  benchmarks/week2_performance.py the full benchmark
================================================================================
```